Name: Siddhant Thalal
Roll no: B21ME066

Lab 01 (Experimental Robotics)

Installation:

1. Downloaded the ubuntu file from the : https://releases.ubuntu.com/jammy/ .
2. Shrink(divided) the disk for using the dual system.
3. Installed the ubuntu.
4. Dual booted the system to run the ubuntu and windows.
5. Installed whole ros2 using the documentation (Debian packages). https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html

Issue faced: during installation of ubuntu the bit locker encryption of windows has stopped the installation so i need to reboot and the toggled off the bit locker button. And then reinstalled Ubuntu and it worked properly.

Also, followed some youtube tutorials for smooth installation.

Tutorials :

Beginner: CLI Tools
In this, I learned how to configure the environment, install turtlesim, and use rqt graph and rqt. I also learned the concepts of Nodes, Topics, Services, Parameters, and Actions, which are used to perform certain tasks. Additionally, I learned how to use rqt console to view logs, launch nodes, record data, and play it back.

Beginner: Client Libraries
I learned how to create workspaces and packages, as well as how to create subscribers and publishers.

Task 01: (Run TurtleSim using linear and angular velocities as the inputs.)

linear velocity =Roll number/10
angular velocity = Roll number/20

Roll no = 66
Linear velocity= 6.6
angular velocity= 3.3

For this task I have gone through all the listed tutorials mentioned in the assignment.

Using the topics in the Beginner CLI tools concepts chapter this task can be done easily.

Steps :

1. Calculated the velocities.
2. Run the source command to access the ros2 commands aka Setup environment.
3. Start turtlesim using the code:
   ros2 run turtlesim turtlesim_node
4. Open new terminal (to publish data to the topic: turtle1/cmd_vel) and using Ros2 topic pub:
5. We can move this as an arc or complete circle using the codes:

   ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 6.6, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 3.3}}"

   Changing the once to rate will lead to making a full circle or rotating turtle.

   ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 6.6, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 3.3}}"

> The difference here is the removal of the `--once` option and the addition of the `--rate 1` option, which tells `ros2 topic pub` to publish the command in a steady stream at 1 Hz. …….from documentation.

In this code I have changed the velocities to the calculated ones.

Here is the link for the youtube video for this task:

To give a linear and angular velocity to turtle in turtle sim ros2.

Task 02:
Write a custom code to send the TurtleSim to the desired position.

Roll number = 66,

x_desired = 66/10 = 6.6,
y_desired = 66/5 = 13.2.

Steps :

1. Created a python script/file using the touch turtle_to_goal.py
2. This file i have used the logic like first calculated the angle desired
3. Then rotated the turtle to that angle within the given threshold after it reached it.
4. Then calculated the distance using the euclidean formula and given a velocity linear to move.
5. Used the controller to decrease the speed when it reaches towards the goal
6. Then stop the turtle when it reaches the goal (within the given threshold) using the break statement.
7. Threshold =0.01,
   For better understanding of the comments in the code.
8. Setup environment.

Add souce to run ros 2 commands.

9. Run the turtlesim node (ros2 run turtlesim turtlesim_node) and in another terminal run the py file.
10. Using python3 file_name.

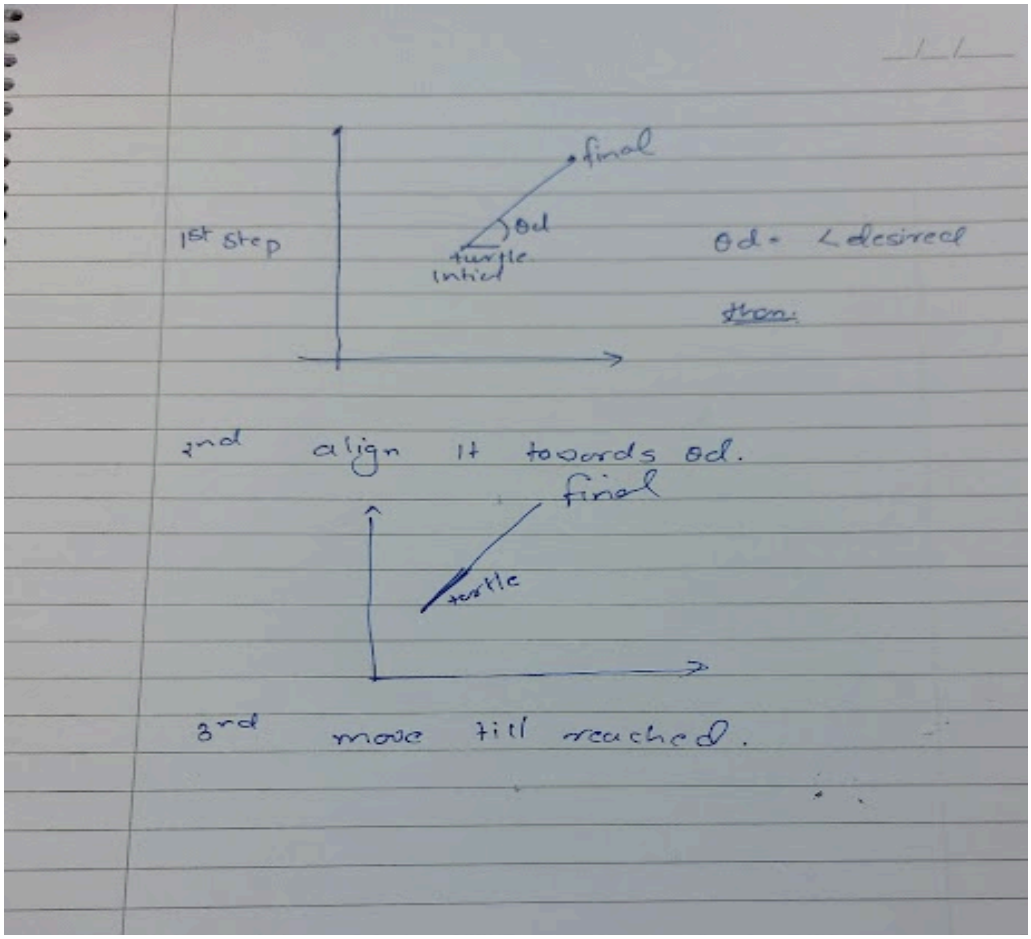
Code:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
import math

class TurtleSimController(Node):
    def __init__(self):
    super().__init__('turtle_controller')
    self.publisher = self.create_publisher(Twist,
'/turtle1/cmd_vel', 10)
    self.subscriber = self.create_subscription(Pose,
'/turtle1/pose', self.pose_callback, 10)
    self.pose = None
    self.x_desired = 6.6
    self.y_desired = 13.2

    def pose_callback(self, msg):
    self.pose = msg

    def move_to_goal(self):
    while self.pose is None:
        rclpy.spin_once(self)

    twist = Twist()
```

```
    # finding the angle / direction for the turtle to
align in the line of the goal
    while True:
        angle_to_goal = math.atan2(self.y_desired -
self.pose.y, self.x_desired - self.pose.x)
        angular_difference = angle_to_goal -
self.pose.theta


        angular_difference =
math.atan2(math.sin(angular_difference),
math.cos(angular_difference))

        angular_speed = 1.0 * angular_difference
        twist.angular.z = angular_speed


        self.publisher.publish(twist)

        # breaking the loop when the turtle has reached to
the mentioned angle near to it within the given threshold

        if abs(angular_difference) < 0.01:
            break

        rclpy.spin_once(self)

    # moving in the straight line as we are aligned
towards the goal

    while True:
        # Calculate the distance to the goal using
euclidean formula
```

```
        distance = math.sqrt((self.x_desired -
self.pose.x) ** 2 + (self.y_desired - self.pose.y) ** 2)




        # given some velocity to the turtle
        linear_speed = 1.0 * distance


        twist.linear.x = linear_speed


        # Publish the Twist message
        self.publisher.publish(twist)

        # If the turtle is close enough to the goal, stop
the movement by breaking the loop
        #here the threshold is 0.01
        if distance < 0.01:
            print("Destination reached!")
            break

        rclpy.spin_once(self)

    # Stop the turtle
    twist.linear.x = 0.0
    twist.angular.z = 0.0
    self.publisher.publish(twist)

def main(args=None):
```

```
    rclpy.init(args=args)
    turtle_controller = TurtleSimController()
    turtle_controller.move_to_goal()
    turtle_controller.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```



This is how I reached the code's logic.

Youtube video for this task:
Python code to send turtle sim to desired coordinates/ position in ros2.

Also the grid size for the turtlesim that i have used is less than the desired output so i have changed the roll no or say input to 30.
Else it moves across the boundary and causes the code to crash.

Values x = 30/10 = 3,
Y = 30/5 = 6 .
Using these values I have done the task.

Also tested for other values or we can change the grid size of the turtle sim.

Lab 02

Task C: Write a custom service and client node, where the client requests basic calculations
(addition, subtraction, multiplication and division) and the server responds.

For this task I have gone through the tutorials.

Steps

1. **Made the srv file with float as inputs and string as operation and float as result**
2. **Then i have written the python codes for the service and the client part**
3. **Client interacts with the person and the input will be given here via client**
4. **Then the server will takes the input using the libraries and make decision via if else statements that what operation to be performed**
5. **The send the response to the client will display on the terminal**

Link for this task

calculator via ros2

Task D:

- Create a ROS package using your name without the “ros2 pkg create” command.

- Create a launch file in this package, which should execute nodes created in task C.

Steps

1. **I have manually created all the directories and the files using touch and mkdir command, added all the dependencies and imported all the needed files.**
2. **The structure must be the same as that of the package.**
3. **Then build the file using colcon build and then it is ready to use.**
4. **The added launch folder and inserted launch.py to access the service and client together and added all the nodes from task c here and then make changes in the setup file to have access outside the folder also.**
5. **Then build all the needed packages and launch the file.**

Link for task :

Screencast from 08 23 2024 11:55:25 PM

Lab 03:

Task A: Explore all the topics subscribed to and published by the TurtleBot and write custom
code to send your TurtleBot on a given desired coordinate.

For this task I have gone through the tutorial and using that I have installed all the required packages.

All the hardware parts were already installed(assembled) and other setup regarding them was already done

Then (Bringup):
Where i have conned to the turtlebot3.

Before this you must connect to the wifi of the robotics lab.

Using the
`ssh ubuntu@{IP_ADDRESS_OF_RASPBERRY_PI}`

Here we have taken the ip address of our bot.

Bring up basic packages to start TurtleBot3 applications.
Here we have changed the model name to waffle_pi.

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ ros2 launch turtlebot3_bringup robot.launch.py
```

When it displays Run! => we are good to go…

We can check the topic list and the services list there and also using this we can check if all this is working properly or not, like proper connections.

```
$ ros2 topic list
$ ros2 service list
```

The console must display the required topic and service else there is something wrong in the installation or connections between bot.

Here using rqt

`plugin` -> `Topics` -> `Topic Monitor`

you can check the frequencies and find battery percentage voltage etc important information.

Using:

```
 $ ros2 run turtlebot3_teleop teleop_keyboard
```

We can control the bot using the keys

- **To move the bot to the desired coordinates**

1. Written the python code.
2. Then done the bringup of the turtlebot3
3. Launch the file python file in new terminal
   Using:
   `python3 file_name`

Codes logic:

`/odom` indicates a message about the odometry of the TurtleBot3. This topic has orientation and position by the encoder data.
This is used to get the position data of bot during the movement.

Divided the movement into three parts

- **First movement in x direction**

- **Second changing the angle 90 towards y direction**
- **Third movement in y direction**

Also added a tolerance and adjusted the speed of the turtlebot3 so that to reduce the error.

Moved in x direction till it reaches the desired coordinate and excluding the tolerance.
And then the angle, same goes for it and similarly the y direction movement.

Video: Move turtlebot3 to desired coordinates.

Video for physical turtlebot3:
Mobile video/ move turtlebot3 to desired coordinates

Task B: Create the Map of the Robotics Lab, save it, and upload the saved Map along with its recorded video.

Connect to the turtlebot3.
The export the waffle_pi model and then do the bringup

Then in new terminal using:

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ ros2 launch turtlebot3_cartographer
cartographer.launch.py
```

Launch the cartographer Rviz where you can see the bot at a certain location.

Using teleop_keyboard you can move the bot around the surrounding

```
$ export TURTLEBOT3_MODEL=waffle_pi
$ ros2 run turtlebot3_teleop teleop_keyboard
```

Move the bot around the boundaries and obstacles, using its sensors like lidar etc it will create a map.

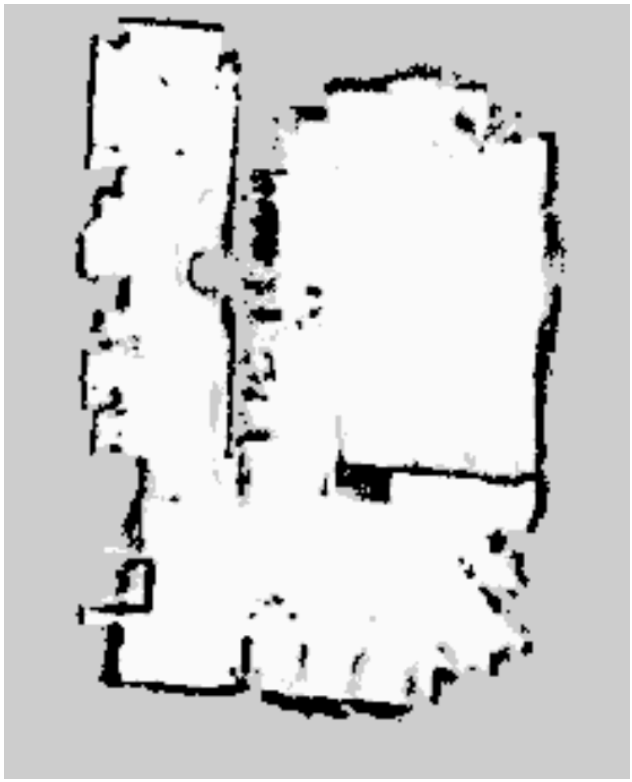
Shown in the video.
how to create map using turtlebot3 and ros2

Also saved the map which we have created

```
$ ros2 run nav2_map_server map_saver_cli -f ~/file_name
```

Shown in the video.
save the map in turtlebot3

The map of lab:



Error faced:
Tf old data : this is arrived because another person also used the same domain id of our bot or say not changed his own so that the RViz can not detect the nodes and launch the map then we have changed our id to 40 and it got solved.

Task C: Navigate three times through the Map that you saved in Task B from any starting position to any final location. Do the screen recording of RViz, and also record the robot.

First save the map according to the last tutorial and then open it using the RViz via the terminal.

Open RViz and using 2d pose estimate Estimate the position of the robot on the map with direction with dragging mouse towards the direction green arrow will appear.

Launch keyboard teleoperation node to precisely locate the robot on the map.

Also stop the teleop terminal by ctrl + c in order to prevent different **cmd_vel** values from being published from multiple nodes during Navigation.

Then using navigation2 Goal in RViz to move the bot to the desired location. It will automatically find its path and will move toward the point and also can adjust angles.

Just click on the desired location and dragging will help you to give an angle.

Also learned about:

Costmap Parameters

- inflation_layer.inflation_radius: Defines the radius around obstacles where the cost is inflated to prevent the robot from getting too close. It should be larger than the robot's radius to ensure safe navigation.

- inflation_layer.cost_scaling_factor: Scales the cost values in the costmap. Increasing this factor decreases the cost near obstacles, encouraging closer paths, while decreasing it encourages the robot to stay farther away.

Navigation using RViz and normal movement:

Navigation through a created map using Rviz and turtlebot3

Navigation mobile video:

Navigation mobile video

Task D:
Do the screen recording of RViz and Gazebo while following the instructions.

For this task I have followed the tutorial thoroughly and followed all the given steps and shown all the subtasks in the video like creating a world in gazebo, simulation or movement in it using teleop_keyboard and also using RViz mapping and navigation like the task C.

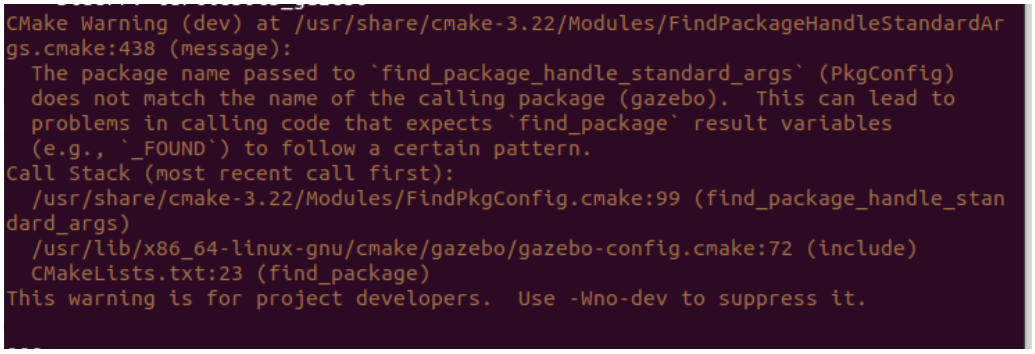
For this using normal installation it is not working so we have used the source installation where we have create the directories and the done the git cloning Then it worked.

```
$ sudo apt remove ros-humble-turtlebot3-msgs
$ sudo apt remove ros-humble-turtlebot3
$ mkdir -p ~/turtlebot3_ws/src
$ cd ~/turtlebot3_ws/src/
```

```
$ git clone -b humble-devel
https://github.com/ROBOTIS-GIT/DynamixelSDK.git
$ git clone -b humble-devel
https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
$ git clone -b humble-devel
https://github.com/ROBOTIS-GIT/turtlebot3.git
$ cd ~/turtlebot3_ws
$ colcon build --symlink-install
$ echo 'source ~/turtlebot3_ws/install/setup.bash' >>
~/.bashrc
$ source ~/.bashrc
```

Error faced: no such directory exists
Then used the above installation.
And it worked.

Warning :



- **Gazebo was not opening so we inserted this line in the bashrc file.**

Steps followed form tutorial shown the video:

Create a map using gazebo and RViz:

how to create a map in Gazebo world

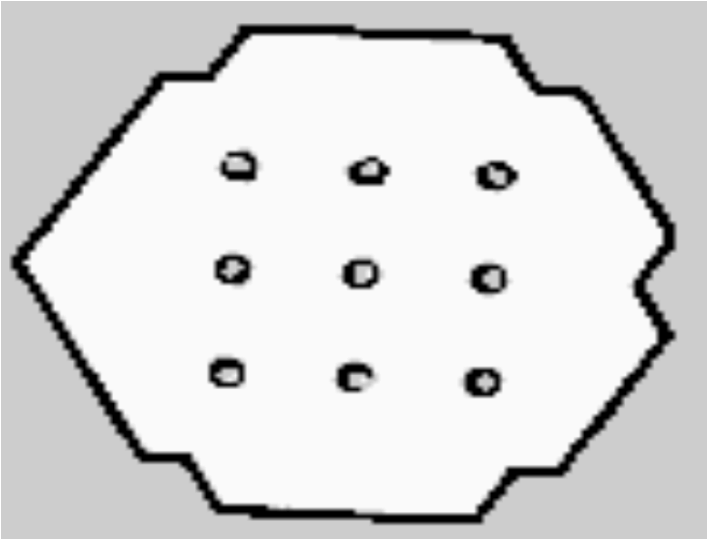
Navigation in that map:

navigation in Gazebo using created map and RViz.

Fake node :

Fake node Simulation.

map :

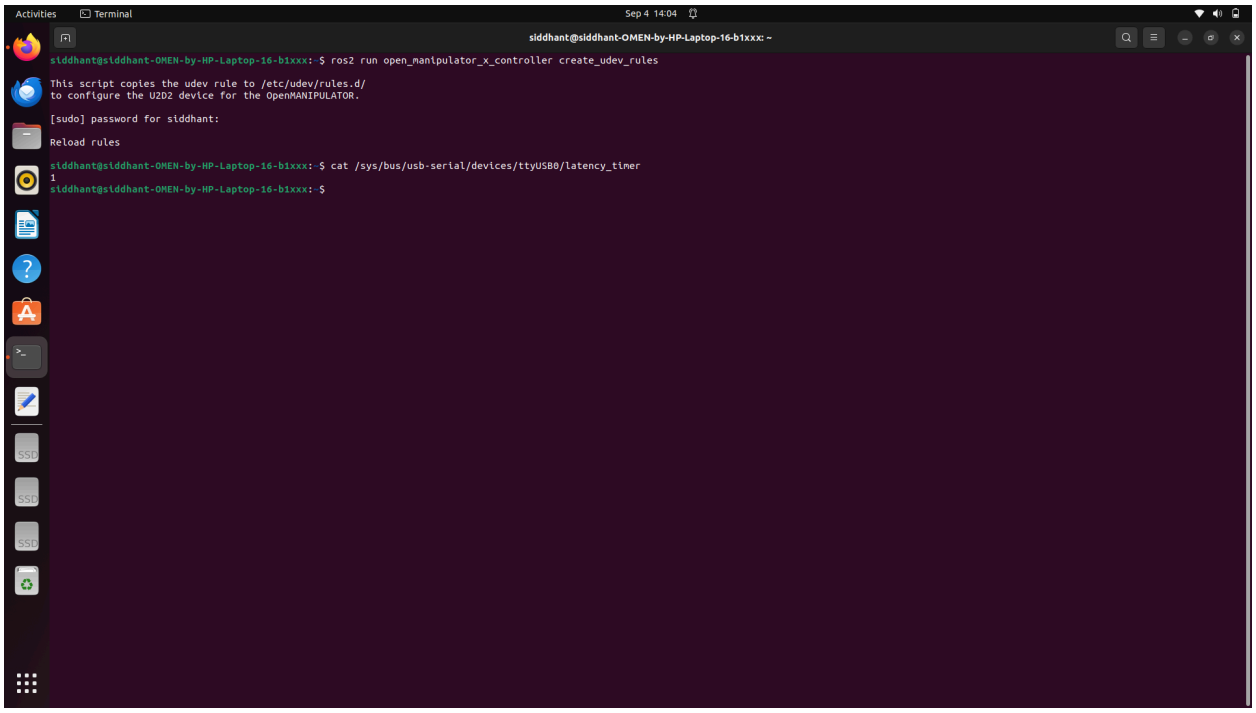


Lab 04 :

1. **Change the USB latency from 16ms to 1ms and save the terminal picture.**

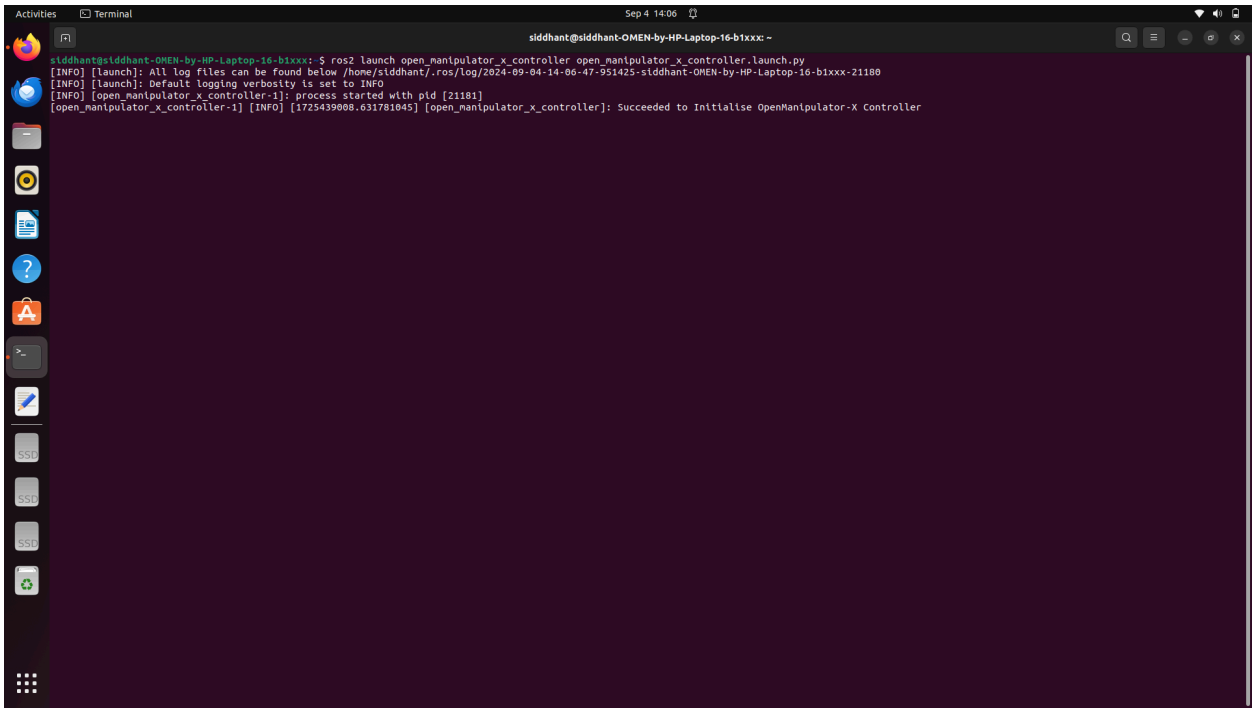
steps : followed all the tutorials.

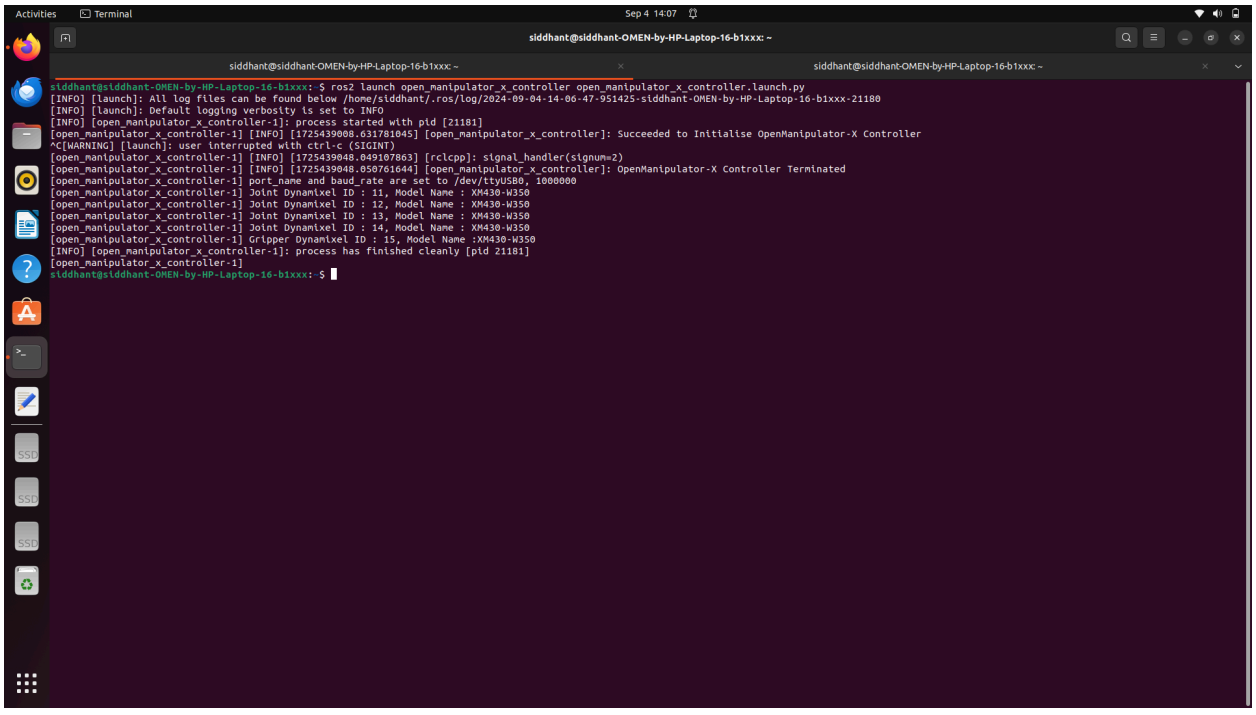
- In this as open manipulator is unavailable for hubble so we used foxy one and made our own directory and did the cloning in there and operated the open manipulator.



Here in the screen we can see that the latency has changed to 1.

2. **Launch the Controller (U2D2) and save the terminal picture.**





- Here in the screenshot we can see that the open manipulator is ready to be controlled.
- And in the next one we can also see the related information of open manipulator

3. **Move the manipulator using keyboard Teleoperation.**

Screen recording :
How to operate open manipulator using keyboard.

Open manipulator recording :

Move the manipulator using keyboard teleoperation.

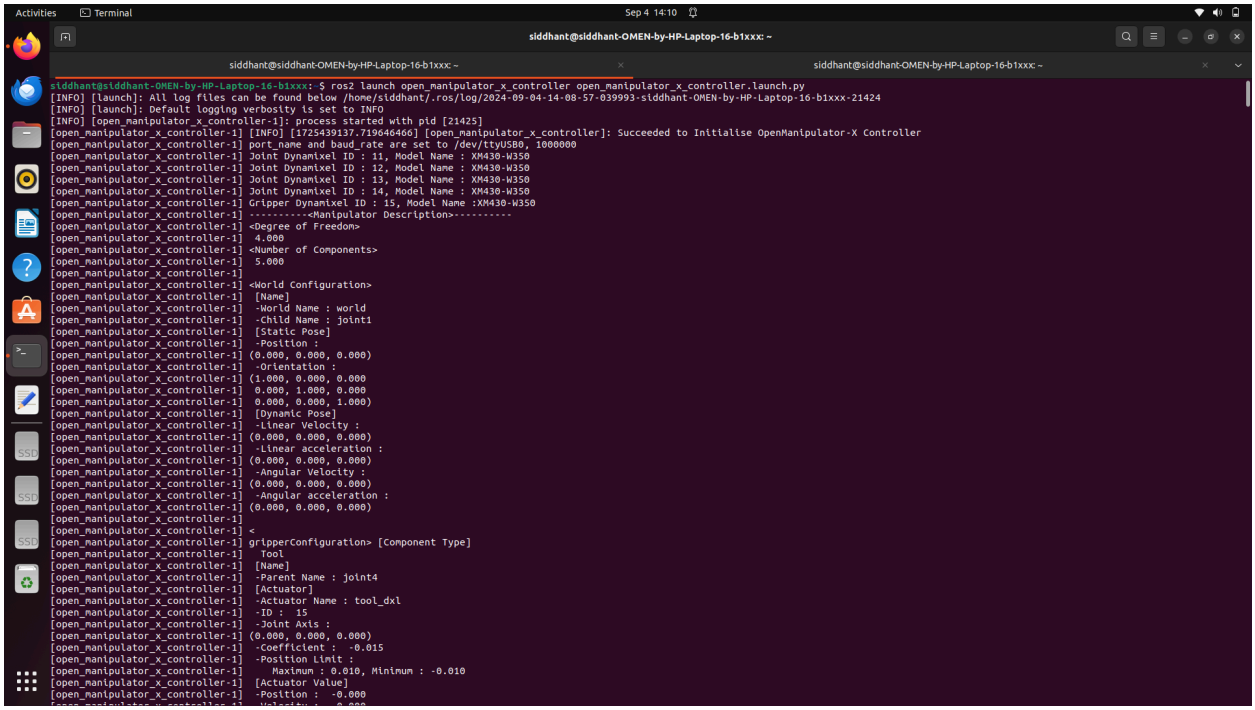
- Here our gripper is not working so we made some changes in the teleop operation python file in the source and added the gripper there and the keys to operate it.

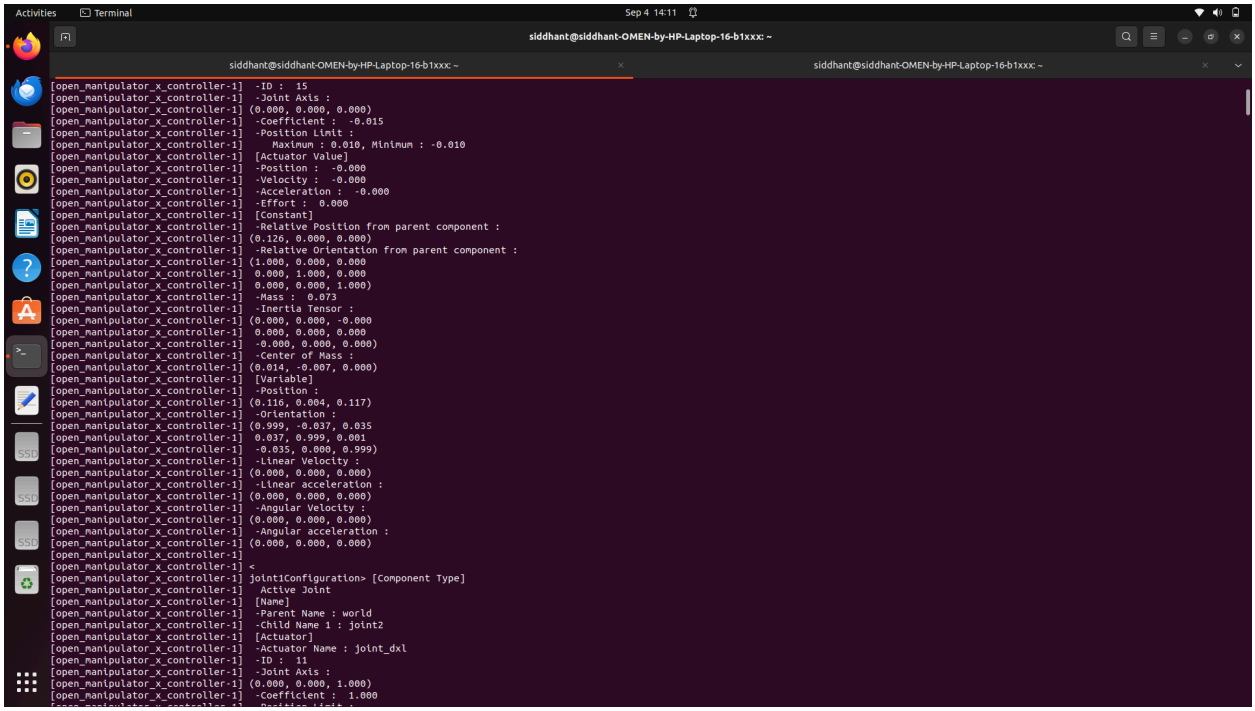
- Then it worked properly.

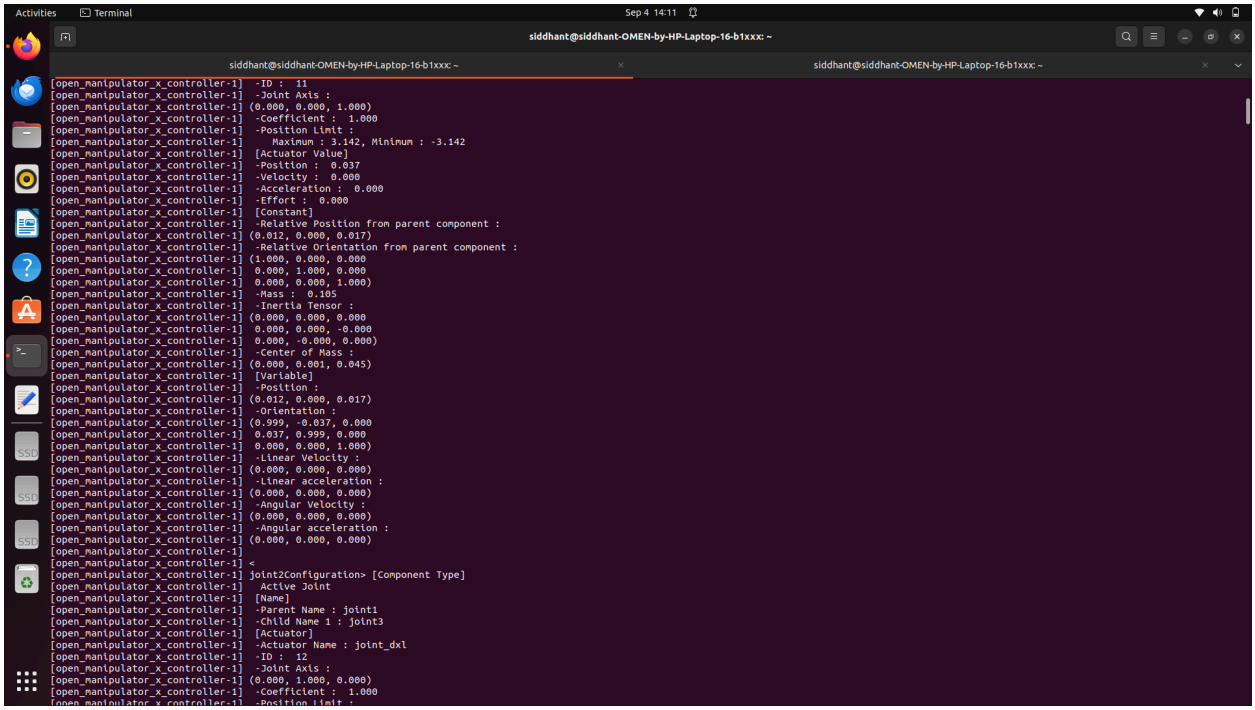
4. **Take a Screenshot of the manipulator Description:**

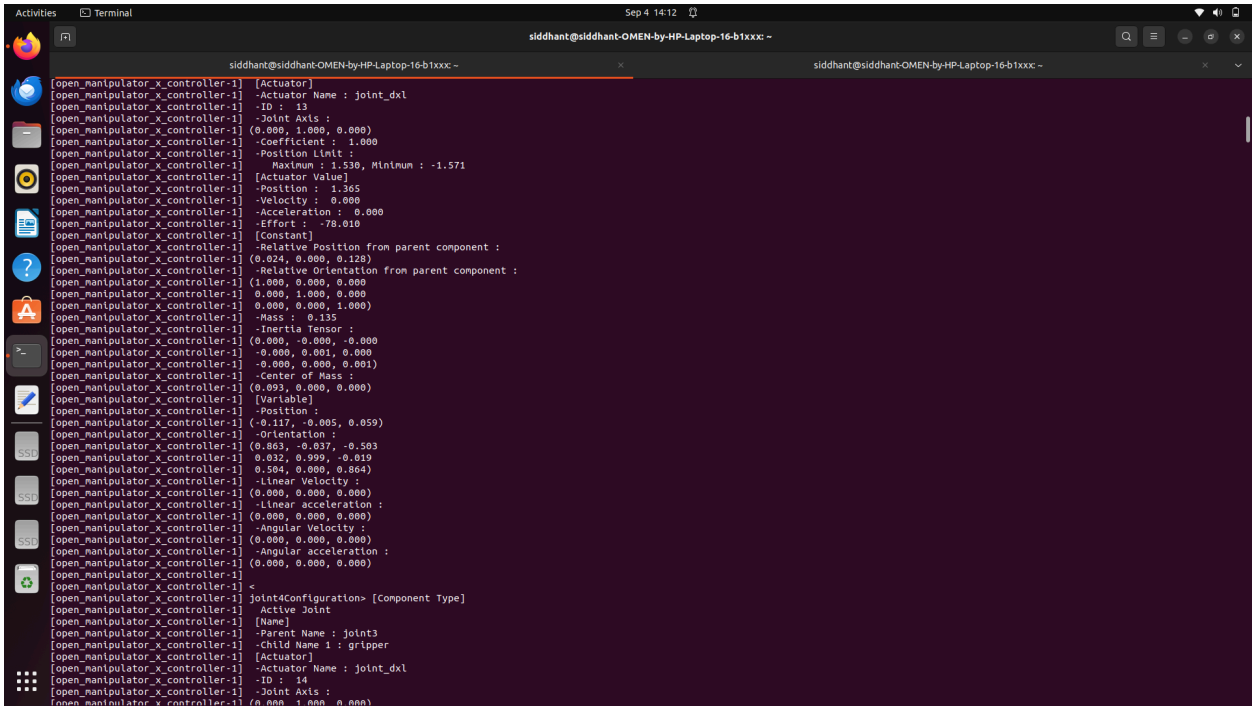
- Followed the tutorial

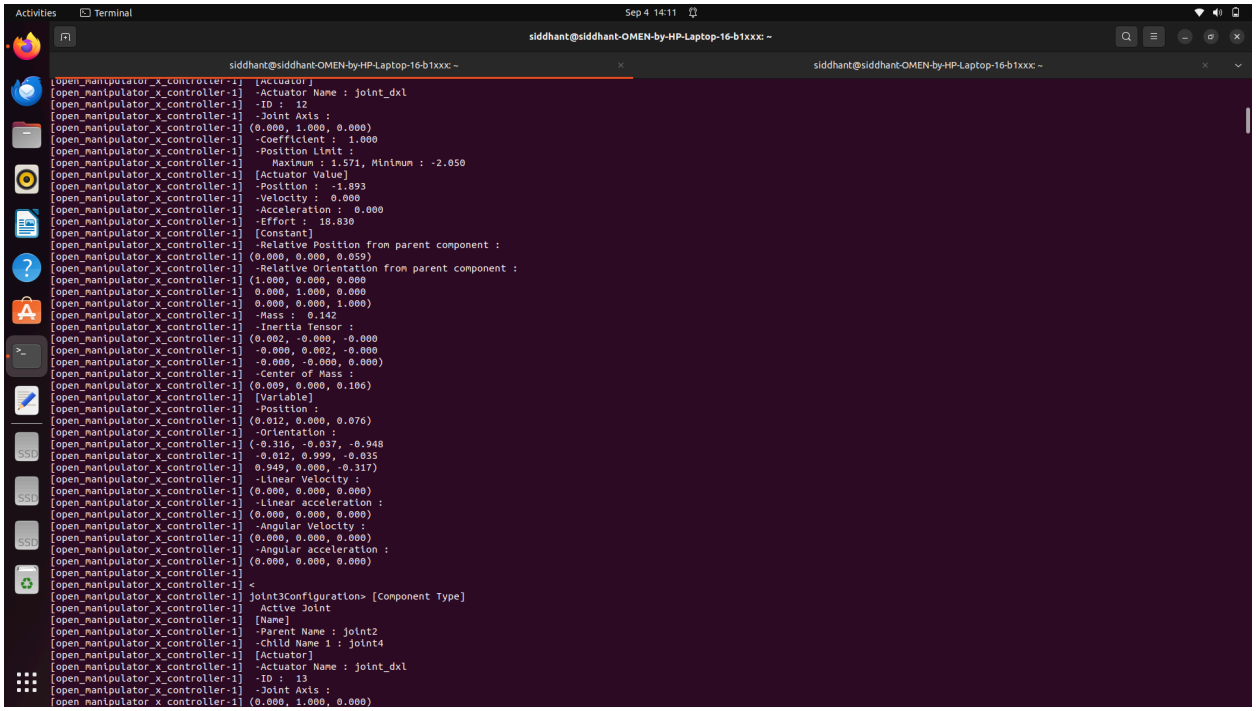
Screenshots are given below and it has all the information or description about the open manipulator.

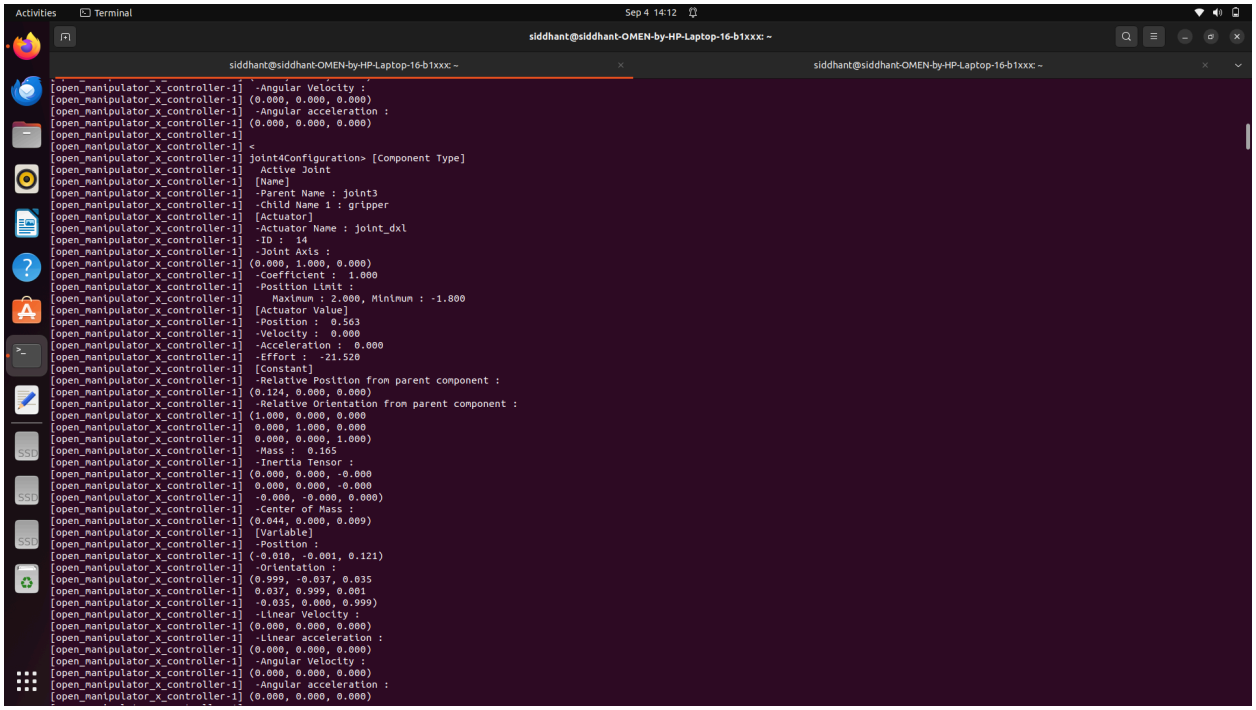

Activities
Terminal
Sep 4 14:10
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx:~$ ros2 launch open_manipulator_x_controller open_manipulator_x_controller.launch.py
[INFO] [launch]: All log files can be found below /home/siddhant/.ros/log/2024-09-04-14-08-57-039993-siddhant-OMEN-by-HP-Laptop-16-b1xxx-21424
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [open_manipulator_x_controller-1]: process started with pid [21425]
[open_manipulator_x_controller-1] [INFO] [1725439137.719646466] [open_manipulator_x_controller]: Succeeded to Initialise OpenManipulator-X Controller
[open_manipulator_x_controller-1] port_name and baud_rate are set to /dev/ttyUSB0, 1000000
[open_manipulator_x_controller-1] Joint Dynamixel ID : 11, Model Name : XM430-W350
[open_manipulator_x_controller-1] Joint Dynamixel ID : 12, Model Name : XM430-W350
[open_manipulator_x_controller-1] Joint Dynamixel ID : 13, Model Name : XM430-W350
[open_manipulator_x_controller-1] Joint Dynamixel ID : 14, Model Name : XM430-W350
[open_manipulator_x_controller-1] Gripper Dynamixel ID : 15, Model Name :XM430-W350
[open_manipulator_x_controller-1] ----------<Manipulator Description>----------
[open_manipulator_x_controller-1] <Degree of Freedom>
[open_manipulator_x_controller-1] 4.000
[open_manipulator_x_controller-1] <Number of Components>
[open_manipulator_x_controller-1] 5.000
[open_manipulator_x_controller-1]
[open_manipulator_x_controller-1] <World Configuration>
[open_manipulator_x_controller-1] [Name]
[open_manipulator_x_controller-1] -World Name : world
[open_manipulator_x_controller-1] -Child Name : joint1
[open_manipulator_x_controller-1] [Static Pose]
[open_manipulator_x_controller-1] -Position :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Orientation :
[open_manipulator_x_controller-1] (1.000, 0.000, 0.000
[open_manipulator_x_controller-1] 0.000, 1.000, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] [Dynamic Pose]
[open_manipulator_x_controller-1] -Linear Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Linear acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1]
[open_manipulator_x_controller-1] <
[open_manipulator_x_controller-1] gripperConfiguration> [Component Type]
[open_manipulator_x_controller-1] Tool
[open_manipulator_x_controller-1] [Name]
[open_manipulator_x_controller-1] -Parent Name : joint4
[open_manipulator_x_controller-1] [Actuator]
[open_manipulator_x_controller-1] -Actuator Name : tool_dxl
[open_manipulator_x_controller-1] -ID : 15
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Coefficient : -0.015
[open_manipulator_x_controller-1] -Position Limit :
[open_manipulator_x_controller-1] Maximum : 0.010, Minimum : -0.010
[open_manipulator_x_controller-1] [Actuator Value]
[open_manipulator_x_controller-1] -Position : -0.000


Activities
Terminal
Sep 4 14:11
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
[open_manipulator_x_controller-1] -ID : 15
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Coefficient : -0.015
[open_manipulator_x_controller-1] -Position Limit :
[open_manipulator_x_controller-1] Maximum : 0.010, Minimum : -0.010
[open_manipulator_x_controller-1] [Actuator Value]
[open_manipulator_x_controller-1] -Position : -0.000
[open_manipulator_x_controller-1] -Velocity : -0.000
[open_manipulator_x_controller-1] -Acceleration : -0.000
[open_manipulator_x_controller-1] -Effort : 0.000
[open_manipulator_x_controller-1] [Constant]
[open_manipulator_x_controller-1] -Relative Position from parent component :
[open_manipulator_x_controller-1] (0.126, 0.000, 0.000)
[open_manipulator_x_controller-1] -Relative Orientation from parent component :
[open_manipulator_x_controller-1] (1.000, 0.000, 0.000
[open_manipulator_x_controller-1] 0.000, 1.000, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Mass : 0.073
[open_manipulator_x_controller-1] -Inertia Tensor :
[open_manipulator_x_controller-1] (0.000, 0.000, -0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 0.000
[open_manipulator_x_controller-1] -0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Center of Mass :
[open_manipulator_x_controller-1] (0.014, -0.007, 0.000)
[open_manipulator_x_controller-1] [Variable]
[open_manipulator_x_controller-1] -Position :
[open_manipulator_x_controller-1] (0.116, 0.004, 0.117)
[open_manipulator_x_controller-1] -Orientation :
[open_manipulator_x_controller-1] (0.999, -0.037, 0.035
[open_manipulator_x_controller-1] 0.037, 0.999, 0.001
[open_manipulator_x_controller-1] -0.035, 0.000, 0.999)
[open_manipulator_x_controller-1] -Linear Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Linear acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1]
[open_manipulator_x_controller-1] <
[open_manipulator_x_controller-1] joint1Configuration> [Component Type]
[open_manipulator_x_controller-1] Active Joint
[open_manipulator_x_controller-1] [Name]
[open_manipulator_x_controller-1] -Parent Name : world
[open_manipulator_x_controller-1] -Child Name 1 : joint2
[open_manipulator_x_controller-1] [Actuator]
[open_manipulator_x_controller-1] -Actuator Name : joint_dxl
[open_manipulator_x_controller-1] -ID : 11
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Coefficient : 1.000


Activities
Terminal
Sep 4 14:11
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
[open_manipulator_x_controller-1] -ID : 11
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Coefficient : 1.000
[open_manipulator_x_controller-1] -Position Limit :
[open_manipulator_x_controller-1] Maximum : 3.142, Minimum : -3.142
[open_manipulator_x_controller-1] [Actuator Value]
[open_manipulator_x_controller-1] -Position : 0.037
[open_manipulator_x_controller-1] -Velocity : 0.000
[open_manipulator_x_controller-1] -Acceleration : 0.000
[open_manipulator_x_controller-1] -Effort : 0.000
[open_manipulator_x_controller-1] [Constant]
[open_manipulator_x_controller-1] -Relative Position from parent component :
[open_manipulator_x_controller-1] (0.012, 0.000, 0.017)
[open_manipulator_x_controller-1] -Relative Orientation from parent component :
[open_manipulator_x_controller-1] (1.000, 0.000, 0.000
[open_manipulator_x_controller-1] 0.000, 1.000, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Mass : 0.105
[open_manipulator_x_controller-1] -Inertia Tensor :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, -0.000
[open_manipulator_x_controller-1] 0.000, -0.000, 0.000)
[open_manipulator_x_controller-1] -Center of Mass :
[open_manipulator_x_controller-1] (0.000, 0.001, 0.045)
[open_manipulator_x_controller-1] [Variable]
[open_manipulator_x_controller-1] -Position :
[open_manipulator_x_controller-1] (0.012, 0.000, 0.017)
[open_manipulator_x_controller-1] -Orientation :
[open_manipulator_x_controller-1] (0.999, -0.037, 0.000
[open_manipulator_x_controller-1] 0.037, 0.999, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Linear Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Linear acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1]
[open_manipulator_x_controller-1] <
[open_manipulator_x_controller-1] joint2Configuration> [Component Type]
[open_manipulator_x_controller-1] Active Joint
[open_manipulator_x_controller-1] [Name]
[open_manipulator_x_controller-1] -Parent Name : joint1
[open_manipulator_x_controller-1] -Child Name 1 : joint3
[open_manipulator_x_controller-1] [Actuator]
[open_manipulator_x_controller-1] -Actuator Name : joint_dxl
[open_manipulator_x_controller-1] -ID : 12
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 1.000, 0.000)
[open_manipulator_x_controller-1] -Coefficient : 1.000
[open_manipulator_x_controller-1] -Position Limit :


Activities
Terminal
Sep 4 14:12
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
[open_manipulator_x_controller-1] [Actuator]
[open_manipulator_x_controller-1] -Actuator Name : joint_dxl
[open_manipulator_x_controller-1] -ID : 13
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 1.000, 0.000)
[open_manipulator_x_controller-1] -Coefficient : 1.000
[open_manipulator_x_controller-1] -Position Limit :
[open_manipulator_x_controller-1] Maximum : 1.530, Minimum : -1.571
[open_manipulator_x_controller-1] [Actuator Value]
[open_manipulator_x_controller-1] -Position : 1.365
[open_manipulator_x_controller-1] -Velocity : 0.000
[open_manipulator_x_controller-1] -Acceleration : 0.000
[open_manipulator_x_controller-1] -Effort : -78.010
[open_manipulator_x_controller-1] [Constant]
[open_manipulator_x_controller-1] -Relative Position from parent component :
[open_manipulator_x_controller-1] (0.024, 0.000, 0.128)
[open_manipulator_x_controller-1] -Relative Orientation from parent component :
[open_manipulator_x_controller-1] (1.000, 0.000, 0.000
[open_manipulator_x_controller-1] 0.000, 1.000, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Mass : 0.135
[open_manipulator_x_controller-1] -Inertia Tensor :
[open_manipulator_x_controller-1] (0.000, -0.000, -0.000
[open_manipulator_x_controller-1] -0.000, 0.001, 0.000
[open_manipulator_x_controller-1] -0.000, 0.000, 0.001)
[open_manipulator_x_controller-1] -Center of Mass :
[open_manipulator_x_controller-1] (0.093, 0.000, 0.000)
[open_manipulator_x_controller-1] [Variable]
[open_manipulator_x_controller-1] -Position :
[open_manipulator_x_controller-1] (-0.117, -0.005, 0.059)
[open_manipulator_x_controller-1] -Orientation :
[open_manipulator_x_controller-1] (0.863, -0.037, -0.503
[open_manipulator_x_controller-1] 0.032, 0.999, -0.019
[open_manipulator_x_controller-1] 0.504, 0.000, 0.864)
[open_manipulator_x_controller-1] -Linear Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Linear acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1]
[open_manipulator_x_controller-1] <
[open_manipulator_x_controller-1] joint4Configuration> [Component Type]
[open_manipulator_x_controller-1] Active Joint
[open_manipulator_x_controller-1] [Name]
[open_manipulator_x_controller-1] -Parent Name : joint3
[open_manipulator_x_controller-1] -Child Name 1 : gripper
[open_manipulator_x_controller-1] [Actuator]
[open_manipulator_x_controller-1] -Actuator Name : joint_dxl
[open_manipulator_x_controller-1] -ID : 14
[open_manipulator_x_controller-1] -Joint Axis :


Activities
Terminal
Sep 4 14:11
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
siddhant@siddhant-OMEN-by-HP-Laptop-16-b1xxx: ~
[open_manipulator_x_controller-1] -Actuator Name : joint_dxl
[open_manipulator_x_controller-1] -ID : 12
[open_manipulator_x_controller-1] -Joint Axis :
[open_manipulator_x_controller-1] (0.000, 1.000, 0.000)
[open_manipulator_x_controller-1] -Coefficient : 1.000
[open_manipulator_x_controller-1] -Position Limit :
[open_manipulator_x_controller-1] Maximum : 1.571, Minimum : -2.050
[open_manipulator_x_controller-1] [Actuator Value]
[open_manipulator_x_controller-1] -Position : -1.893
[open_manipulator_x_controller-1] -Velocity : 0.000
[open_manipulator_x_controller-1] -Acceleration : 0.000
[open_manipulator_x_controller-1] -Effort : 18.830
[open_manipulator_x_controller-1] [Constant]
[open_manipulator_x_controller-1] -Relative Position from parent component :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.059)
[open_manipulator_x_controller-1] -Relative Orientation from parent component :
[open_manipulator_x_controller-1] (1.000, 0.000, 0.000
[open_manipulator_x_controller-1] 0.000, 1.000, 0.000
[open_manipulator_x_controller-1] 0.000, 0.000, 1.000)
[open_manipulator_x_controller-1] -Mass : 0.142
[open_manipulator_x_controller-1] -Inertia Tensor :
[open_manipulator_x_controller-1] (0.002, -0.000, -0.000
[open_manipulator_x_controller-1] -0.000, 0.002, -0.000
[open_manipulator_x_controller-1] -0.000, -0.000, 0.000)
[open_manipulator_x_controller-1] -Center of Mass :
[open_manipulator_x_controller-1] (0.009, 0.000, 0.106)
[open_manipulator_x_controller-1] [Variable]
[open_manipulator_x_controller-1] -Position :
[open_manipulator_x_controller-1] (0.012, 0.000, 0.076)
[open_manipulator_x_controller-1] -Orientation :
[open_manipulator_x_controller-1] (-0.316, -0.037, -0.948
[open_manipulator_x_controller-1] -0.012, 0.999, -0.035
[open_manipulator_x_controller-1] 0.949, 0.000, -0.317)
[open_manipulator_x_controller-1] -Linear Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Linear acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular Velocity :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1] -Angular acceleration :
[open_manipulator_x_controller-1] (0.000, 0.000, 0.000)
[open_manipulator_x_controller-1]
[open_manipulator_x_controller-1] <
[open_manipulator_x_controller-1] joint3Configuration> [Component Type]
[open_manipulator_x_controller-1] Active Joint
[open_manipulator_x_controller-1] [Name]
[open_manipulator_x_controller-1] -Parent Name : joint2
[open_manipulator_x_controller-1] -Child Name 1 : joint4
[open_manipulator_x_controller-1] [Actuator]
[open_manipulator_x_controller-1] -Actuator Name : joint_dxl
[open_manipulator_x_controller-1] -ID : 13
[open_manipulator_x_controller-1] -Joint Axis :



5. **Launch the manipulator in Rviz and control it using a joint state publisher or keyboard.**

Video for this :

Launch the manipulator in Rviz and control it using a joint state publisher or …

- All the followed steps are shown in the video.

- The open manipulator and RViz bot both run together here.
- We are using Rvix just for visualization.

6. **Read this step very carefully as it is required for practical 4. (make a note of all topics and nodes)**

Monitored Topics:

- Used the rqt tool to observe key ROS topics such as /states, /joint_states, and /kinematics_pose, etc which provide information on the robot's status, joint states, and end-effector pose.

Utilized Services:

- Interacted with ROS services to create and manage trajectories, control actuators, and move the robot’s tool.

Topics

1. /states
    - Type: open_manipulator_msgs/msg/OpenManipulatorState
    - Description: Provides status updates on OpenMANIPULATOR-X, including actuator state and movement status.
2. /joint_states
    - Type: sensor_msgs/msg/JointState
    - Description: Reports joint states, including angles, velocities, and efforts.
3. /kinematics_pose
    - Type: open_manipulator_msgs/msg/KinematicsPose
    - Description: Indicates the position and orientation of the end-effector in task space.

Services

1. /goal_joint_space_path
    - Type: open_manipulator_msgs/srv/SetJointPosition
    - Description: Sets a joint trajectory based on target angles and duration.
2. /goal_joint_space_path_to_kinematics_pose
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a joint trajectory to achieve a specified kinematics pose in task space.
3. /goal_joint_space_path_to_kinematics_position

    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a joint trajectory based on target position only.
4. /goal_joint_space_path_to_kinematics_orientation
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a joint trajectory based on target orientation only.
5. /goal_task_space_path
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a task space trajectory with both position and orientation.
6. /goal_task_space_path_position_only
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a task space trajectory based on position only.
7. /goal_task_space_path_orientation_only
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a task space trajectory based on orientation only.
8. /goal_joint_space_path_from_present
    - Type: open_manipulator_msgs/srv/SetJointPosition
    - Description: Sets a trajectory from the current joint state.
9. /goal_task_space_path_from_present
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a trajectory from the current task space pose.
10. /goal_task_space_path_from_present_position_only
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a trajectory from the current position only.
11. /goal_task_space_path_from_present_orientation_only
    - Type: open_manipulator_msgs/srv/SetKinematicsPose
    - Description: Sets a trajectory from the current orientation only.
12. /goal_tool_control
    - Type: open_manipulator_msgs/srv/SetJointPosition
    - Description: Moves the tool of OpenMANIPULATOR-X.
13. /set_actuator_state
    - Type: open_manipulator_msgs/srv/SetActuatorState
    - Description: Enables or disables the actuators.
14. /goal_drawing_trajectory
    - Type: open_manipulator_msgs/srv/SetDrawingTrajectory

- Description: Creates predefined drawing trajectories (e.g., circle, heart).

Lab 05

Task 1: Custom Forward Kinematics Node:

Step 1
Find out the dh parameters for the open manipulator

Link	a i	Alpha i	d i	Theta i
1	0	90	0.077	theta1
2	0.128	0	0	theta2+90
3	0.148	0	0	theta3-90
4	0.126	0	0	theta4

Step 2 forward kinematics
FORWARD KINEMATICS: THE DENAVIT-HARTENBERG CONVENTION
I Went through this pdf for better understanding.

- **Homogeneous transformation Ai (product of four basic transformation)**

$$
\begin{aligned}
A_i &= R_{z,\theta_i}\,\mathrm{Trans}_{z,d_i}\,\mathrm{Trans}_{x,a_i}\,R_{x,\alpha_i} \qquad (3.10)\\
&= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\\
&= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned}
$$

Made this matrix for all links.

Code for this :

```
def dh_transformation_matrix(self, theta, alpha, a, d):
    """Generate a Denavit-Hartenberg transformation
matrix."""
    return np.array([
        [np.cos(theta), -np.sin(theta) * np.cos(alpha),
np.sin(theta) * np.sin(alpha), a * np.cos(theta)],
        [np.sin(theta), np.cos(theta) * np.cos(alpha),
-np.cos(theta) * np.sin(alpha), a * np.sin(theta)],
        [0, np.sin(alpha), np.cos(alpha), d],
        [0, 0, 0, 1]
    ])
```

Step 3
For the transformation matrix:

The T-matrices are thus given by

$$T_1^0 = A_1. \tag{3.24}$$

$$T_2^0 = A_1 A_2 = \begin{bmatrix} c_{12} & -s_{12} & 0 & a_1 c_1 + a_2 c_{12} \\ s_{12} & c_{12} & 0 & a_1 s_1 + a_2 s_{12} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{3.25}$$

Use the all matrices defined above and do the dot product or say matrix multiplication.

Step 4

Getting the coordinates:

Notice that the first two entries of the last column of T_2^0 are the x and y components of the origin O_2 in the base frame; that is,

$$\begin{aligned} x &= a_1 c_1 + a_2 c_{12} \\ y &= a_1 s_1 + a_2 s_{12} \end{aligned} \tag{3.26}$$

From this we got the coordinates x , y and z
Like:

```
    pose_msg = Pose()
    pose_msg.position.x = end_effector_transform[0, 3]
    pose_msg.position.y = end_effector_transform[1, 3]
    pose_msg.position.z = end_effector_transform[2, 3]
```

Here `end_effector_transform` is the transformation matrix.

Video:

https://youtu.be/LuguJ9aXfhM

Final code:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose
import numpy as np

class KinematicsNode(Node):
    def __init__(self):
    super().__init__('kinematics_node')
    self.pose_publisher = self.create_publisher(Pose,
'/end_effector_position', 10)
    self.process_joint_angles()
```

```
    def process_joint_angles(self):
    # Obtain joint angles from user input
    angles = [float(input(f"Please enter joint angle
θ{i+1} (in radians): ")) for i in range(4)]

    # Calculate the forward kinematics based on these
angles
    end_effector_transform =
self.calculate_forward_kinematics(*angles)

    # Convert transformation matrix to pose message
    pose_msg = Pose()
    pose_msg.position.x = end_effector_transform[0, 3]
    pose_msg.position.y = end_effector_transform[1, 3]
    pose_msg.position.z = end_effector_transform[2, 3]

    # Convert rotation matrix to quaternion
    quaternion =
self.convert_matrix_to_quaternion(end_effector_transform[:3
, :3])
    pose_msg.orientation.x, pose_msg.orientation.y,
pose_msg.orientation.z, pose_msg.orientation.w = quaternion

    # Publish the pose of the end-effector
    self.pose_publisher.publish(pose_msg)
    self.get_logger().info(f'Published end-effector pose:
{pose_msg}')

    def dh_transformation_matrix(self, theta, alpha, a,
d):
    """Generate a Denavit-Hartenberg transformation
matrix."""
```

```
    return np.array([
        [np.cos(theta), -np.sin(theta) * np.cos(alpha),
np.sin(theta) * np.sin(alpha), a * np.cos(theta)],
        [np.sin(theta), np.cos(theta) * np.cos(alpha),
-np.cos(theta) * np.sin(alpha), a * np.sin(theta)],
        [0, np.sin(alpha), np.cos(alpha), d],
        [0, 0, 0, 1]
    ])

    def calculate_forward_kinematics(self, q1, q2, q3,
q4):
    """Compute the forward kinematics given joint
angles."""
    transforms = [
        self.dh_transformation_matrix(q1, np.pi / 2, 0,
0.077),
        self.dh_transformation_matrix(q2 + np.pi / 2, 0,
0.128, 0),
        self.dh_transformation_matrix(q3 - np.pi / 2, 0,
0.148, 0),
        self.dh_transformation_matrix(q4, 0, 0.126, 0)
    ]
    result_transform = np.eye(4)
    for transform in transforms:
        result_transform = np.dot(result_transform,
transform)
    return result_transform

    def convert_matrix_to_quaternion(self,
rotation_matrix):
    """Convert a rotation matrix to a quaternion."""
    trace = np.trace(rotation_matrix)
    if trace > 0:
```

```
        s = np.sqrt(trace + 1.0) * 2  # s = 4 * qw
        qw = 0.25 * s
        qx = (rotation_matrix[2, 1] - rotation_matrix[1,
2]) / s
        qy = (rotation_matrix[0, 2] - rotation_matrix[2,
0]) / s
        qz = (rotation_matrix[1, 0] - rotation_matrix[0,
1]) / s
    elif (rotation_matrix[0, 0] > rotation_matrix[1, 1])
and (rotation_matrix[0, 0] > rotation_matrix[2, 2]):
        s = np.sqrt(1.0 + rotation_matrix[0, 0] -
rotation_matrix[1, 1] - rotation_matrix[2, 2]) * 2  # s = 4
* qx
        qw = (rotation_matrix[2, 1] - rotation_matrix[1,
2]) / s
        qx = 0.25 * s
        qy = (rotation_matrix[0, 1] + rotation_matrix[1,
0]) / s
        qz = (rotation_matrix[0, 2] + rotation_matrix[2,
0]) / s
    elif rotation_matrix[1, 1] > rotation_matrix[2, 2]:
        s = np.sqrt(1.0 + rotation_matrix[1, 1] -
rotation_matrix[0, 0] - rotation_matrix[2, 2]) * 2  # s = 4
* qy
        qw = (rotation_matrix[0, 2] - rotation_matrix[2,
0]) / s
        qx = (rotation_matrix[0, 1] + rotation_matrix[1,
0]) / s
        qy = 0.25 * s
        qz = (rotation_matrix[1, 2] + rotation_matrix[2,
1]) / s
    else:
        s = np.sqrt(1.0 + rotation_matrix[2, 2] -
```

```
rotation_matrix[0, 0] - rotation_matrix[1, 1]) * 2  # s = 4
* qz
        qw = (rotation_matrix[1, 0] - rotation_matrix[0,
1]) / s
        qx = (rotation_matrix[0, 2] + rotation_matrix[2,
0]) / s
        qy = (rotation_matrix[1, 2] + rotation_matrix[2,
1]) / s
        qz = 0.25 * s

    return qx, qy, qz, qw

def main(args=None):
    rclpy.init(args=args)
    node = KinematicsNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Task 2: Visualize (Simulation)

Steps :

- **First we made a list of joint positions and then sent to the topic and then using tf we get the end effector positions and for many joint positions we can iterate over them.**
- **Then we calculated the difference between the position via forward kinematics and the received ones.**

code:

```
import rclpy
from rclpy.node import Node
from trajectory_msgs.msg import JointTrajectory,
JointTrajectoryPoint
from sensor_msgs.msg import JointState
from tf2_msgs.msg import TFMessage
import numpy as np
from tf2_ros import TransformException
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener

class CombinedNode(Node):
    def __init__(self):
    super().__init__('combined_node')

    # Publisher to send joint trajectory to Gazebo
    self.publisher =
self.create_publisher(JointTrajectory,
'/joint_trajectory_controller/joint_trajectory', 10)

    # Subscription to listen to the joint states from
Gazebo
    self.subscription_joint_state =
self.create_subscription(
        JointState,
        '/joint_states',
        self.joint_state_callback,
        10
    )

    # Subscription to the tf topic for dynamic transforms
    self.subscription_tf = self.create_subscription(
```

```
            TFMessage,
            '/tf',
            self.tf_callback,
            10
        )

        # Define multiple sets of joint angles (including
gripper angle)
        self.joint_position_sets = [
            [0.0, 0.0, 0.0, 0.0, 0.0],  # Set 1
            [0.4, 0.3, 0.3, 0.4, 0.0],  # Set 2
            [1.5, 0.0, 0.0, 0.0, 0.0],  # Set 3
            [-1.5, -0.3, 0.3, -0.1, 0.0]  # Set 4
        ]

        self.current_set_index = 0  # Track the current set of
angles
        self.waiting_for_joint_states = False  # Flag to track
if waiting for joint states

        # Store the transformations from TF
        self.transformations = {}

        # Start the timer to publish joint angles
        self.timer = self.create_timer(1.0,
self.publish_joint_trajectory)
        self.expected_joint_positions = None  # To store the
expected joint positions

        # Declare parameters for the two frames you want to
listen to
        self.from_frame = self.declare_parameter('from_frame',
'end_effector_link').get_parameter_value().string_value
```

```
    self.to_frame = self.declare_parameter('to_frame',
'world').get_parameter_value().string_value

    # Create the tf buffer and listener
    self.tf_buffer = Buffer()
    self.tf_listener = TransformListener(self.tf_buffer,
self)

    def publish_joint_trajectory(self):
    if self.waiting_for_joint_states:
        self.get_logger().info('Waiting for joint
states...')
        return

    if self.current_set_index >=
len(self.joint_position_sets):
        self.get_logger().info("All joint position sets
have been processed.")
        self.destroy_node()
        return

    # Get the current set of joint positions
    joint_positions =
self.joint_position_sets[self.current_set_index]
    self.get_logger().info(f'Publishing joint trajectory
for set {self.current_set_index + 1}')

    # Create and publish the JointTrajectory message
    msg = JointTrajectory()
    msg.joint_names = ['joint1', 'joint2', 'joint3',
'joint4', 'gripper']
    point = JointTrajectoryPoint()
    point.positions = joint_positions
```

```
        point.time_from_start.sec = 1
        msg.points.append(point)
        self.publisher.publish(msg)

        # Store the expected joint positions for comparison
        self.expected_joint_positions = joint_positions
        self.waiting_for_joint_states = True

        def joint_state_callback(self, msg: JointState):
        if self.expected_joint_positions is None:
            return

        # Extract the received joint positions from the
JointState message
        received_positions =
[msg.position[msg.name.index(name)] for name in ['joint1',
'joint2', 'joint3', 'joint4', 'gripper'] if name in
msg.name]

        # Compare the received joint positions with the
expected ones
        if self.compare_positions(received_positions,
self.expected_joint_positions):
            self.get_logger().info(f'Successfully matched set
{self.current_set_index + 1}')
            if len(self.transformations) == 6:  # Ensure all
necessary transforms are received
                self.calculate_and_compare_end_effector()
            else:
                self.get_logger().error('Not all required TF
transforms are available.')

            # Move to the next set
```

```
            self.current_set_index += 1
            self.expected_joint_positions = None
            self.waiting_for_joint_states = False

        def tf_callback(self, msg: TFMessage):
        # Extract and store the transformations from TF
        for transform in msg.transforms:
            key = (transform.header.frame_id,
transform.child_frame_id)
            self.transformations[key] = [
                transform.transform.translation.x,
                transform.transform.translation.y,
                transform.transform.translation.z
            ]

        def calculate_and_compare_end_effector(self):
        # Calculate the end-effector position using FK based
on joint angles
        try:
            joint_angles =
self.joint_position_sets[self.current_set_index]

            # Perform Forward Kinematics using DH parameters
            end_effector_position_fk =
self.forward_kinematics(joint_angles)
            self.get_logger().info(f'End Effector Position
from FK: {end_effector_position_fk}')

            # Fetch the transform from TF
            try:
                t = self.tf_buffer.lookup_transform(
                    self.to_frame,
                    self.from_frame,
```

```
                    rclpy.time.Time()
                )
                end_effector_position_tf = [
                    t.transform.translation.x,
                    t.transform.translation.y,
                    t.transform.translation.z
                ]
                self.get_logger().info(f'End Effector
Position from TF: {end_effector_position_tf}')

                # Compare FK and TF end-effector positions
                diff =
np.linalg.norm(np.array(end_effector_position_fk) -
np.array(end_effector_position_tf))
                self.get_logger().info(f'Difference in End
Effector Position: {diff}')

            except TransformException as ex:
                self.get_logger().error(f'Could not transform
{self.to_frame} to {self.from_frame}: {ex}')

        except KeyError as e:
            self.get_logger().error(f'Missing TF transform:
{e}')

    def forward_kinematics(self, joint_angles):
    # Define the DH parameters (These values depend on
your specific robot)
    # Modify angles according to DH conventions
    joint_angles[1] += np.deg2rad(90)
    joint_angles[2] -= np.deg2rad(90)

    # DH Parameters for OpenMANIPULATOR-X: [theta, d, a,
```

```
alpha] for each joint
    DH_params = [
        [joint_angles[0], 0.077, 0.0, np.pi / 2],
        [joint_angles[1], 0.0, 0.128, 0.0],
        [joint_angles[2], 0.0, 0.148, 0.0],
        [joint_angles[3], 0.0, 0.126, 0.0]
    ]

    T_shift = np.array([
        [1, 0, 0, 0.012],
        [0, 1, 0, 0],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

    # Start with the shift matrix
    T = T_shift

    for params in DH_params:
        theta, d, a, alpha = params
        T = np.dot(T, self.dh_transformation(theta, d, a,
alpha))

    # Extract the position of the end-effector from the
final transformation matrix
    end_effector_position = T[0:3, 3]  # The translation
part of the transformation matrix
    return end_effector_position

    def dh_transformation(self, theta, d, a, alpha):
    # Create a transformation matrix using DH parameters
    return np.array([
        [np.cos(theta), -np.sin(theta) * np.cos(alpha),
```

```
np.sin(theta) * np.sin(alpha), a * np.cos(theta)],
          [np.sin(theta), np.cos(theta) * np.cos(alpha),
-np.cos(theta) * np.sin(alpha), a * np.sin(theta)],
          [0, np.sin(alpha), np.cos(alpha), d],
          [0, 0, 0, 1]
     ])

     def compare_positions(self, received_positions,
expected_positions, tolerance=0.1):
     return all(abs(received_positions[i] -
expected_positions[i]) < tolerance for i in range(5))

def main(args=None):
     rclpy.init(args=args)
     node = CombinedNode()
     rclpy.spin(node)
     node.destroy_node()
     rclpy.shutdown()

if __name__ == '__main__':
     main()
```

Video link:

simulation for open manipulator

Lab 06

Task 1

Done the calculation for inverse kinematics of open manipulator using some references and youtube videos .

First converted into a single planer of 3R robot by using the calculation the of the angle using tan inverse y/x and then make the local coordinate system for the 3R robot and then find the joint angles

Shown elbow up and elbow down positions.

Reference : Inverse Kinematics of Planar Manipulators (2R and 3R)

Code:

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point
from std_msgs.msg import Float32MultiArray
import numpy as np
import math
import threading

class OpenManipulatorIKNode(Node):
    def __init__(self):
    super().__init__('open_manipulator_ik_node')

    # Subscriber for end effector position
    self.create_subscription(Point,
'end_effector_position', self.position_callback, 10)
    # Publisher for joint angles
    self.joint_pub =
self.create_publisher(Float32MultiArray, 'joint_positions',
10)
```

```
    self.joint_angles = Float32MultiArray()

    # Start a thread for input handling
    self.input_thread =
threading.Thread(target=self.handle_input)
    self.input_thread.start()

    def position_callback(self, msg, phi):
    x = msg.x
    y = msg.y
    z = msg.z

    # Call the IK function
    joint_positions = self.inverse_kinematics(x, y, z,
phi)

    # Unpack joint positions
    theta1, theta2, theta3, theta4, theta2_new,
theta3_new, theta4_new = joint_positions

    # Publish the joint angles (you can choose which set
to publish)
    self.joint_angles.data = [theta1, theta2, theta3,
theta4]  # Publishing the elbow-up configuration
    # To publish the elbow-down configuration, you can
uncomment the next line:
    # self.joint_angles.data = [theta1, theta2_new,
theta3_new, theta4_new]

    self.joint_pub.publish(self.joint_angles)

    # Print the joint angles
```

```
    print(f"Computed joint angles (elbow-up):
θ1={theta1:.3f}, θ2={theta2:.3f}, θ3={theta3:.3f},
θ4={theta4:.3f}")
    print(f"Computed joint angles (elbow-down):
θ1={theta1:.3f}, θ2={theta2_new:.3f}, θ3={theta3_new:.3f},
θ4={theta4_new:.3f}")

    def inverse_kinematics(self, x, y, z, phi):
    d1 = 0.077  # Link 1 offset
    d2 = 0.128  # Link 2 length
    d3 = 0.148  # Link 3 length
    d4 = 0.126  # Link 4 length

    # Calculate θ1
    theta1 = math.atan2(y, x)

    x_new = x / math.cos(theta1)  # Convert the global
axis to local axis
    A = (x_new - d4 * math.cos(phi))
    B = (z - d1 - d4 * math.sin(phi))

    # Calculate θ3 using cosine rule
    D = A**2 + B**2 - d2**2 - d3**2
    l = round(D, 5)
    m = round(2 * d2 * d3, 5)

    if m == 0:  # Avoid division by zero
        raise ValueError("Invalid configuration: division
by zero in calculating theta3.")

    theta3 = math.acos(l / m)  # Elbow-up solution for θ3
    theta3_new = -1 * math.acos(l / m)  # Elbow-down
solution for θ3
```

```
    # Calculate θ2 for both elbow-up and elbow-down
solutions
    a = d3 * math.sin(theta3)
    b = d2 + d3 * math.cos(theta3)
    theta2 = math.atan2(B, A) - math.atan2(a, b)

    a_new = d3 * math.sin(theta3_new)
    b_new = d2 + d3 * math.cos(theta3_new)
    theta2_new = math.atan2(B, A) - math.atan2(a_new,
b_new)

    # Adjust θ2 to make the natural position 90 degrees
(π/2 radians)
    theta2 -= math.pi / 2
    theta2_new -= math.pi / 2
    theta3 += math.pi / 2
    theta3_new += math.pi / 2

    # Calculate θ4 to ensure the desired end-effector
orientation
    theta4 = phi - theta2 - theta3
    theta4_new = phi - theta2_new - theta3_new

    return theta1, theta2, theta3, theta4, theta2_new,
theta3_new, theta4_new

    def handle_input(self):
    """Handle user input for the end effector position."""
    while True:
        try:
            x = float(input("Enter the value of x: "))
            y = float(input("Enter the value of y: "))
```

```
                z = float(input("Enter the value of z: "))
                phi = float(input("Enter the value of phi:
"))  # Accept phi from user input
                point_msg = Point(x=x, y=y, z=z)
                self.position_callback(point_msg, phi)  #
Call the position callback with phi
            except ValueError:
                print("Invalid input. Please enter numeric
values.")
            except KeyboardInterrupt:
                print("Input handling stopped.")
                break

def main(args=None):
    rclpy.init(args=args)
    ik_node = OpenManipulatorIKNode()
    rclpy.spin(ik_node)
    ik_node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Code without Node only inverse kinematics:

```
import math

def calculate_inverse_kinematics(x, y, z, phi):
    # Link parameters
    d1 = 0.077  # Link 1 offset
    d2 = 0.128  # Link 2 length
    d3 = 0.148  # Link 3 length
```

```
    d4 = 0.126  # Link 4 length

    # Calculate θ1
    theta1 = math.atan2(y, x)


    x_new = x / math.cos(theta1) # convert the global axis
to local axis
    A = (x_new - d4 * math.cos(phi))
    B = (z - d1 - d4 * math.sin(phi))

    # Calculate θ3 using cosine rule
    D = A**2 + B**2 - d2**2 - d3**2
    l = round(D, 5)
    m = round(2 * d2 * d3, 5)

    if abs(l/m) > 1:  # Check for possible errors in case
the configuration is unreachable
    raise ValueError("The target position is out of the
robot's reach.")

    theta3 = math.acos(l / m)  # Elbow-up solution for θ3

    # _new means elbow-down solutions
    theta3_new = -1 * math.acos(l / m)  # Elbow-down
solution for θ3

    # Calculate θ2 for both elbow-up and elbow-down
solutions
    a = d3 * math.sin(theta3)
    b = d2 + d3 * math.cos(theta3)
    theta2 = math.atan2(B, A) - math.atan2(a, b)
```

```
    a_new = d3 * math.sin(theta3_new)
    b_new = d2 + d3 * math.cos(theta3_new)
    theta2_new = math.atan2(B, A) - math.atan2(a_new,
b_new)

    # Adjust θ2 to make the natural position 90 degrees
(π/2 radians)
    theta2 -= math.pi / 2
    theta2_new -= math.pi / 2
    theta3 += math.pi / 2
    theta3_new += math.pi / 2
    # Calculate θ4 to ensure the desired end-effector
orientation
    theta4 = phi - theta2 - theta3
    theta4_new = phi - theta2_new - theta3_new

    return theta1, theta2, theta3, theta4, theta2_new,
theta3_new, theta4_new

def main():
    # Get user input for end-effector position and
orientation
    x = float(input("Enter the x-coordinate of the
end-effector: "))
    y = float(input("Enter the y-coordinate of the
end-effector: "))
    z = float(input("Enter the z-coordinate of the
end-effector: "))
    phi = float(input("Enter the orientation phi (θ2 + θ3
+ θ4) in radians: "))

    # Calculate inverse kinematics
    try:
```

```
    theta1, theta2, theta3, theta4, theta2_new,
theta3_new, theta4_new = calculate_inverse_kinematics(x, y,
z, phi)

    # Print the joint angles
    print(f"Calculated Joint Angles (Elbow-Up Solution):")
    print(f"θ1 = {theta1:.4f} radians")
    print(f"θ2 = {theta2:.4f} radians")
    print(f"θ3 = {theta3:.4f} radians")
    print(f"θ4 = {theta4:.4f} radians")

print(f"_____________________________________________")
    print(f"\nCalculated Joint Angles (Elbow-Down
Solution):")
    print(f"θ2 = {theta2_new:.4f} radians")
    print(f"θ3 = {theta3_new:.4f} radians")
    print(f"θ4 = {theta4_new:.4f} radians")

    except ValueError as e:
    print(e)

if __name__ == "__main__":
    main()
```

Video link:
https://youtu.be/Njkauj_ubUI

Task 2

Here we have to iterate over some number of values of x , y , z , phi and then send the joint angles to the open manipulator and receive the x, y, x, phi values

and relate them.

1. **We have imputed the coordinates.**
2. **Then using inverse kinematics got the joint angles.**
3. **Used the joint angles as input for open manipulator.**
4. **Simulated it.**
5. **Go the coordinates from Gazebo joint states.**
6. **Compared them.**

And the error is nearly equal to zero. I have highlighted it in the video.

Code:

```
import rclpy
from rclpy.node import Node
from trajectory_msgs.msg import JointTrajectory,
JointTrajectoryPoint
from sensor_msgs.msg import JointState
from tf2_msgs.msg import TFMessage
import numpy as np
from tf2_ros import TransformException
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
import math

class CombinedNode(Node):
    def __init__(self):
    super().__init__('combined_node')
```

```
        # Publisher to send joint trajectory to Gazebo
        self.publisher =
self.create_publisher(JointTrajectory,
'/joint_trajectory_controller/joint_trajectory', 10)

        # Subscription to listen to the joint states from
Gazebo
        self.subscription_joint_state =
self.create_subscription(
            JointState,
            '/joint_states',
            self.joint_state_callback,
            10
        )

        # Subscription to the tf topic for dynamic transforms
        self.subscription_tf = self.create_subscription(
            TFMessage,
            '/tf',
            self.tf_callback,
            10
        )

        # Initialize joint position sets
        self.joint_position_sets = []
        self.current_set_index = 0  # Track the current set of
angles
        self.waiting_for_joint_states = False  # Flag to track
if waiting for joint states
        self.transformations = {}  # Store the transformations
from TF

        # Start the timer to publish joint angles
```

```
    self.timer = self.create_timer(1.0,
self.publish_joint_trajectory)
    self.expected_joint_positions = None  # To store the
expected joint positions

    # Declare parameters for the two frames you want to
listen to
    self.from_frame = self.declare_parameter('from_frame',
'end_effector_link').get_parameter_value().string_value
    self.to_frame = self.declare_parameter('to_frame',
'world').get_parameter_value().string_value

    # Create the tf buffer and listener
    self.tf_buffer = Buffer()
    self.tf_listener = TransformListener(self.tf_buffer,
self)

    # Ask user for end-effector position and orientation
    self.get_user_input()

    def get_user_input(self):
    # Get user input for end-effector position and
orientation
    try:
        x = float(input("Enter the x-coordinate of the
end-effector: "))
        y = float(input("Enter the y-coordinate of the
end-effector: "))
        z = float(input("Enter the z-coordinate of the
end-effector: "))
        phi = float(input("Enter the orientation phi (θ2 +
θ3 + θ4) in radians: "))
        self.calculate_inverse_kinematics(x, y, z, phi)
```

```
    except ValueError:
        self.get_logger().error("Invalid input! Please
enter numeric values.")

    def calculate_inverse_kinematics(self, x, y, z, phi):
    # Link parameters
    d1 = 0.077  # Link 1 offset
    d2 = 0.128  # Link 2 length
    d3 = 0.148  # Link 3 length
    d4 = 0.126  # Link 4 length

    # Calculate θ1
    theta1 = math.atan2(y, x)
    x_new = x / math.cos(theta1)  # Convert the global
axis to local axis
    A = (x_new - d4 * math.cos(phi))
    B = (z - d1 - d4 * math.sin(phi))

    # Calculate θ3 using cosine rule
    D = A**2 + B**2 - d2**2 - d3**2
    l = round(D, 5)
    m = round(2 * d2 * d3, 5)

    if abs(l/m) > 1:  # Check for possible errors in case
the configuration is unreachable
        self.get_logger().error("The target position is
out of the robot's reach.")
        return

    theta3 = math.acos(l / m)  # Elbow-up solution for θ3
    theta3_new = -1 * math.acos(l / m)  # Elbow-down
solution for θ3
```

```
    # Calculate θ2 for both elbow-up and elbow-down
solutions
    a = d3 * math.sin(theta3)
    b = d2 + d3 * math.cos(theta3)
    theta2 = math.atan2(B, A) - math.atan2(a, b)

    a_new = d3 * math.sin(theta3_new)
    b_new = d2 + d3 * math.cos(theta3_new)
    theta2_new = math.atan2(B, A) - math.atan2(a_new,
b_new)

    # Adjust θ2 to make the natural position 90 degrees
(π/2 radians)
    theta2 -= math.pi / 2
    theta2_new -= math.pi / 2
    theta3 += math.pi / 2
    theta3_new += math.pi / 2

    # Calculate θ4 to ensure the desired end-effector
orientation
    theta4 = phi - theta2 - theta3
    theta4_new = phi - theta2_new - theta3_new

    # Store the calculated joint angles in the joint
position sets
        # 0.0 for gripper, if applicable
    self.joint_position_sets.append([theta1,
-1*theta2_new, -1*theta3_new, -1*theta4_new, 0.0])  #
Elbow-down solution

    # Log the calculated angles
    self.get_logger().info(f"Calculated Joint Angles
(Elbow-Up Solution):")
```

```
    self.get_logger().info(f"θ1 = {theta1:.4f} radians")
    self.get_logger().info(f"θ2 = {theta2:.4f} radians")
    self.get_logger().info(f"θ3 = {theta3:.4f} radians")
    self.get_logger().info(f"θ4 = {theta4:.4f} radians")

self.get_logger().info(f"_____________________________________
_____________")
    self.get_logger().info(f"Calculated Joint Angles
(Elbow-Down Solution):")
    self.get_logger().info(f"θ2 = {theta2_new:.4f}
radians")
    self.get_logger().info(f"θ3 = {theta3_new:.4f}
radians")
    self.get_logger().info(f"θ4 = {theta4_new:.4f}
radians")

    def publish_joint_trajectory(self):
    if self.waiting_for_joint_states:
        self.get_logger().info('Waiting for joint
states...')
        return

    if self.current_set_index >=
len(self.joint_position_sets):
        self.get_logger().info("All joint position sets
have been processed.")
        self.destroy_node()
        return

    # Get the current set of joint positions
    joint_positions =
self.joint_position_sets[self.current_set_index]
    self.get_logger().info(f'Publishing joint trajectory
```

```
for set {self.current_set_index + 1}')

    # Create and publish the JointTrajectory message
    msg = JointTrajectory()
    msg.joint_names = ['joint1', 'joint2', 'joint3',
'joint4', 'gripper']
    point = JointTrajectoryPoint()
    point.positions = joint_positions
    point.time_from_start.sec = 1
    msg.points.append(point)
    self.publisher.publish(msg)

    # Store the expected joint positions for comparison
    self.expected_joint_positions = joint_positions
    self.waiting_for_joint_states = True

    def joint_state_callback(self, msg: JointState):
    if self.expected_joint_positions is None:
        return

    # Extract the received joint positions from the
JointState message
    received_positions =
[msg.position[msg.name.index(name)] for name in ['joint1',
'joint2', 'joint3', 'joint4', 'gripper'] if name in
msg.name]

    # Compare the received joint positions with the
expected ones
    if self.compare_positions(received_positions,
self.expected_joint_positions):
        self.get_logger().info(f'Successfully matched set
{self.current_set_index + 1}')
```

```
          if len(self.transformations) == 6:  # Ensure all
necessary transforms are received
               self.calculate_and_compare_end_effector()
          else:
               self.get_logger().error('Not all required TF
transforms are available.')

          # Move to the next set
          self.current_set_index += 1
          self.expected_joint_positions = None
          self.waiting_for_joint_states = False

     def tf_callback(self, msg: TFMessage):
     # Extract and store the transformations from TF
     for transform in msg.transforms:
          key = (transform.header.frame_id,
transform.child_frame_id)
          self.transformations[key] = [
               transform.transform.translation.x,
               transform.transform.translation.y,
               transform.transform.translation.z
          ]

     def calculate_and_compare_end_effector(self):
     # Calculate the end-effector position using FK based
on joint angles
     try:
          joint_angles =
self.joint_position_sets[self.current_set_index]
          end_effector_position_fk =
self.forward_kinematics(joint_angles)
```

```
          # Fetch the transform from TF
          try:
               t = self.tf_buffer.lookup_transform(
                    self.to_frame,
                    self.from_frame,
                    rclpy.time.Time()
               )
               end_effector_position_tf = [
                    t.transform.translation.x,
                    t.transform.translation.y,
                    t.transform.translation.z
               ]
               self.get_logger().info(f'End Effector
Position from TF: {end_effector_position_tf}')



          except TransformException as ex:
               self.get_logger().error(f'Could not transform
{self.to_frame} to {self.from_frame}: {ex}')

     except KeyError as e:
          self.get_logger().error(f'Missing TF transform:
{e}')

     def forward_kinematics(self, joint_angles):
     # Define the DH parameters
     joint_angles[1] += np.deg2rad(90)
     joint_angles[2] -= np.deg2rad(90)

     DH_params = [
          [joint_angles[0], 0.077, 0.0, np.pi / 2],
          [joint_angles[1], 0.0, 0.128, 0.0],
```

```
            [joint_angles[2], 0.0, 0.148, 0.0],
            [joint_angles[3], 0.0, 0.126, 0.0]
        ]

        T_shift = np.array([
            [1, 0, 0, 0.012],
            [0, 1, 0, 0],
            [0, 0, 1, 0],
            [0, 0, 0, 1]
        ])

        # Start with the shift matrix
        T = T_shift

        for params in DH_params:
            theta, d, a, alpha = params
            T = np.dot(T, self.dh_transformation(theta, d, a,
alpha))

        # Extract the position of the end-effector from the
final transformation matrix
        end_effector_position = T[0:3, 3]
        return end_effector_position

        def dh_transformation(self, theta, d, a, alpha):
        # Create a transformation matrix using DH parameters
        return np.array([
            [np.cos(theta), -np.sin(theta) * np.cos(alpha),
np.sin(theta) * np.sin(alpha), a * np.cos(theta)],
            [np.sin(theta), np.cos(theta) * np.cos(alpha),
-np.cos(theta) * np.sin(alpha), a * np.sin(theta)],
            [0, np.sin(alpha), np.cos(alpha), d],
            [0, 0, 0, 1]
```

```
        ])

    def compare_positions(self, received_positions,
expected_positions, tolerance=0.1):
        return all(abs(received_positions[i] -
expected_positions[i]) < tolerance for i in range(5))

def main(args=None):
    rclpy.init(args=args)
    node = CombinedNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Video :

https://youtu.be/WLMbwqcHvSk

LAB 07

Tasks to Complete:

- **Task 1: Develop a package for joint and end-effector control (hardware).**
- **Task 2: Create a custom node for trajectory generation.**
- **Task 3: Visualize the trajectory in RViz.**
- **Task 4: Implement trajectory control for OpenManipulator-X.**

Task 1: Package for Joint and End-effector Control (Hardware)

Task 2: Custom Node for Trajectory Generation

- **Node Setup: A custom ROS 2 node, *TrajectoryGenerationNode*, was created to handle cycloidal trajectory generation.**
- **Inputs and Parameters: The node accepts parameters such as initial and final positions of the end-effector, total time for the motion, and the current time to compute the intermediate trajectory points.**
- **Cycloidal Trajectory Calculation: The cycloidal trajectory equation was applied to ensure smooth transitions along the path with minimal jerk.**
- **Inverse Kinematics: An inverse kinematics function was integrated to translate each calculated end-effector position into the corresponding joint angles.**
- **Trajectory Publishing: The joint angles were published as *JointTrajectory* messages at each time step, facilitating smooth robot motion.**
- **Execution and Verification: The node was tested by observing the robot following the cycloidal path smoothly, transitioning accurately from the start position to the target end position.**

Task 3: Visualization of Trajectory in RViz

- **Node Setup for RViz Visualization: The *TrajectoryGenerationNode* was modified to visualize the entire trajectory in RViz. The trajectory was computed over the entire duration of the motion.**
- **Computing Full Trajectory: Intermediate positions along the trajectory were calculated by discretizing the total time and applying the cycloidal formula to determine each point.**
- **Joint Angle Conversion: For each position, inverse kinematics was used to derive the corresponding joint angles.**
- **Trajectory Publishing for RViz: The entire sequence of joint angles was published as a *JointTrajectory* message, enabling RViz to visualize the smooth motion along the computed path.**
- **Testing: The visualized trajectory in RViz was checked to confirm it accurately represented the cycloidal path, verifying smooth movement from the initial to final positions.**

Task 4: Trajectory Control of OpenManipulator-X

- **Defining the Trajectory Path: A smooth trajectory for the manipulator's end-effector was generated using the cycloidal equation.**
- **Calculating Joint Angles: The inverse kinematics was applied to each point along the trajectory to compute the corresponding joint angles.**
- **Publishing Joint Commands to Hardware: A complete set of joint angles was published to the OpenManipulator-X's joint publisher, enabling the manipulator to follow the trajectory on the hardware.**
- **Verification: The movement of the manipulator was observed to ensure that it followed the cycloidal path accurately, with synchronized joint movements throughout.**

LAB 08

Tasks to Complete:

- **Task 1: Derive the Jacobian matrix.**
- **Task 2: Develop a custom node for computing the Jacobian.**
- **Task 3: Control the end-effector of OpenManipulator-X in Gazebo.**

- **Task 4: Control the end-effector of OpenManipulator-X on hardware.**

Task 1: Derivation of the Jacobian

- **Kinematic Modeling: The Denavit-Hartenberg (DH) parameters were defined for the manipulator to model the joint and link configurations, including joint angles, link lengths, twists, and offsets.**
- **Forward Kinematics: Using the DH parameters, the transformation matrices for each joint were calculated to determine the overall position and orientation of the end-effector in the base frame.**
- **Jacobian Derivation: The Jacobian matrix was derived by computing the partial derivatives of the end-effector's position and orientation with respect to the joint angles.**
- **Singular Configurations: Singularities were identified by computing the determinant of the Jacobian matrix, noting cases where the determinant was zero, indicating a loss of movement capability in certain directions.**
- **Conclusion: The Jacobian matrix was summarized, and singular configurations were highlighted where the manipulator's degrees of freedom were reduced.**

Task 2: Custom Node for Jacobian

- **Define DH Parameters: The DH parameters, including joint angles (θ1, θ2, θ3, θ4), link lengths, and twists, were defined.**
- **Transformation Matrices: Individual transformation matrices for each joint were computed using the DH parameters.**
- **End-Effector Position: The position of the end-effector was determined by multiplying the transformation matrices.**
- **Jacobian Structure:**
    - **Linear Velocity: Calculated using the cross-product between the joint axis and position difference.**
    - **Angular Velocity: Determined using unit vectors along each joint axis.**
- **Jacobian Matrix: The Jacobian matrix was composed by combining linear and angular velocities.**
- **Code Implementation: A custom Python/ROS node was written to**

compute the Jacobian matrix for given joint angles.

- **Testing: The Jacobian was verified by testing with known joint configurations.**

Task 3: End-Effector Control of OpenManipulator-X in Gazebo

- **Gazebo Setup: The Gazebo environment was configured with the OpenManipulator-X model and ROS 2 controllers.**
- **Jacobian Calculation: The Jacobian matrix for the current joint configuration was computed.**
- **Inverse Jacobian Application: The inverse Jacobian method was used to calculate the required change in joint angles to reach the target end-effector position.**
- **Trajectory Commands: The joint trajectory commands were published to move the manipulator toward the desired end-effector pose.**
- **Verification: After each movement, the actual end-effector position was compared with the target using the *tf_listener*, ensuring accurate movement. This iterative process allowed the manipulator to reach the desired position smoothly.**

Task 4: End-Effector Control of OpenManipulator-X on Hardware

- **Hardware Setup: The OpenManipulator-X hardware was connected to ROS 2, with the appropriate drivers and controllers configured.**
- **Publishing Joint Commands: Joint commands, based on the inverse Jacobian method, were published to the manipulator's joints to achieve the target end-effector pose.**
- **Iterative Method: To prevent abrupt movements, the inverse Jacobian method was applied iteratively.**
- **Real-Time Feedback: The manipulator's end-effector pose was monitored in real-time to verify accuracy.**
- **Hardware Constraints: The method was adjusted to respect joint limits and ensure safe operation.**
- **Testing: The hardware was tested to ensure that the manipulator reached the desired pose with precision.**

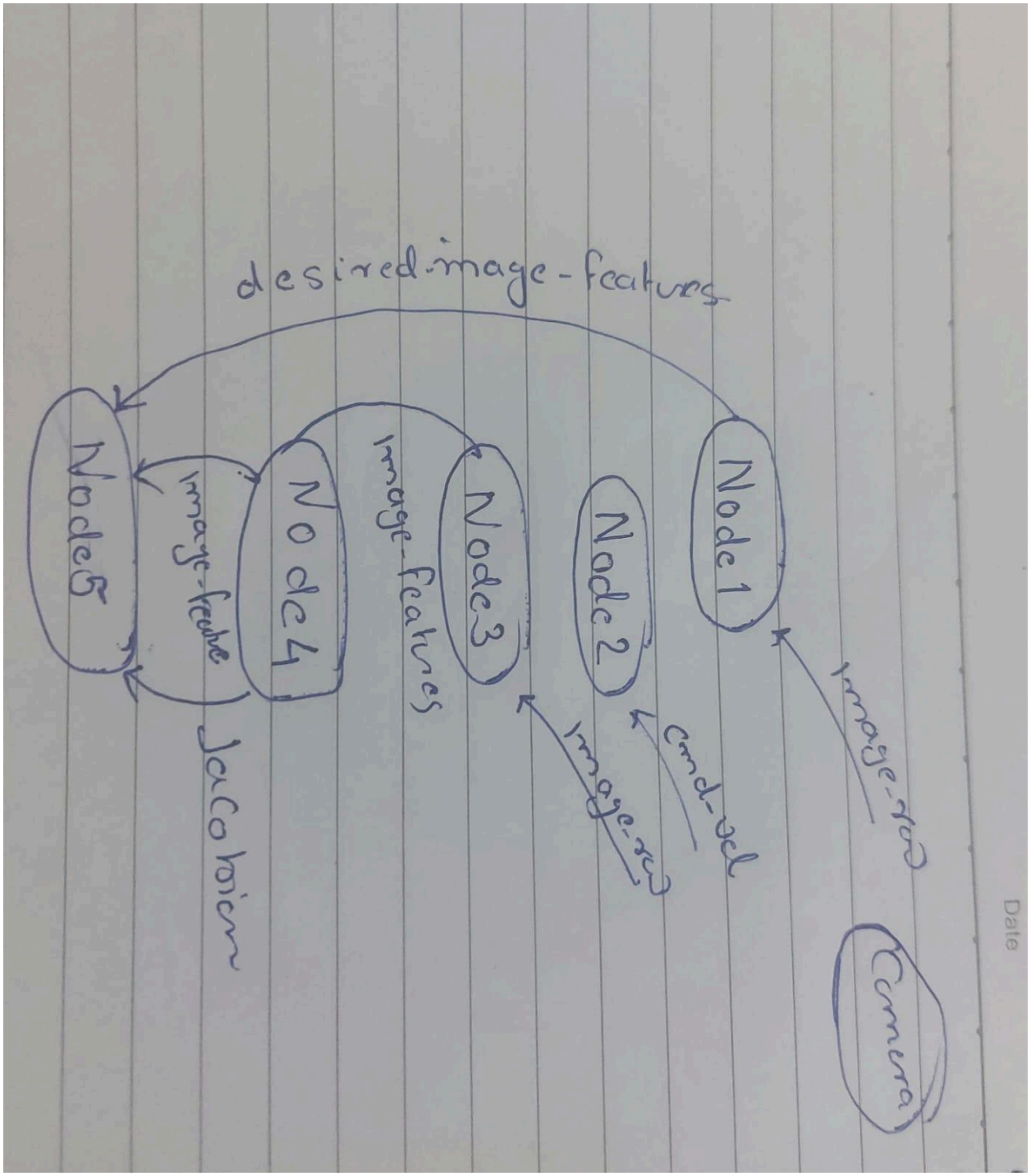
LAB 09 - 10

The OpenCV (cv2) library was utilized to detect circles on the box. The parameters for the functions used in the detection process were fine-tuned to suit the specific case and optimize performance.

To achieve the desired outcome, multiple ROS2 nodes were implemented:

1. **Feature Array Node(s): These nodes were responsible for extracting the feature arrays from the image data, including the detection of circles, and adjusting their initial positions as required.**
2. **Current Feature Array Node: This node was used to retrieve the current feature arrays, representing the most up-to-date information regarding the detected features on the box.**
3. **Jacobian Calculation Node: This node computed the Jacobian matrix, which is critical for understanding the relationship between the feature array and the robot's end-effector position.**
4. **Velocity Calculation Node: The final node utilized the Jacobian matrix and the current feature array to compute the required velocities for the TurtleBot, enabling it to adjust its movements based on the feature information and Jacobian calculations.**

By coordinating these nodes, the system was designed to dynamically adjust the TurtleBot’s movements in real-time, based on the detected features and their relationship to the robot's position.



Code and video files are here.

LAB 7-10